

Stock Volatility Prediction for Risk Management

Syed Islam¹ 

¹Department of Natural Sciences, University of Texas at Austin, Austin, TX, USA

*Corresponding author:

Abstract

This paper investigates the use of machine learning models to forecast short-term stock return volatility for large-cap U.S. equities. Accurate volatility forecasts are critical for applications such as portfolio risk management, position sizing, and regulatory capital calculations. Using a five-year panel of daily OHLCV data for the 500 largest companies by market capitalization, I constructed realized volatility targets based on rolling windows of log returns and engineer a set of price- and volume-based features, including lagged returns, lagged volatility, high-low ranges, moving averages, and liquidity indicators. I framed volatility prediction as a supervised regression problem and compare several approaches: naive historical-volatility baselines, linear regression models, and tree-based ensemble methods such as random forests and gradient boosting. Models are trained and evaluated using a strictly time-ordered train-validation-test split, with performance measured by mean absolute error and root mean squared error. I further analyzed feature importance to identify which market signals are most predictive of future volatility. The results demonstrate that appropriately tuned tree-based models can provide more accurate volatility forecasts than simple historical baselines, while maintaining interpretability useful for practical risk management.

Keywords: Stock volatility; machine learning; risk management; time series forecasting; financial markets.

1 Introduction

Looking at financial markets today, we can see that they are highly volatile, and managing this volatility is central to risk management, portfolio allocation, and derivative pricing. In this paper, volatility refers to the variability of daily stock returns, measured as the standard deviation of recent log returns over a rolling window. Accurate volatility forecasts allow investors and risk managers to size positions, set margins, and evaluate downside risk more efficiently. However, volatility is time-varying, clustered, and driven by complex interactions among market variables, which makes forecasting challenging. These complexities have motivated the use of machine learning methods that can capture nonlinear relationships beyond traditional econometric models.

Traditional volatility models, such as GARCH-type specifications and historical realized-volatility benchmarks (Zhu et al., 2023), have been widely adopted in practice. More recent studies have applied machine learning methods to forecast realized volatility (Zhang et al., 2024), often demonstrating improvements over linear benchmarks when rich feature

sets are available. These studies typically employ high-frequency or panel data and evaluate a range of nonlinear models, including tree-based methods and neural networks. Despite these advances, little attention has been paid to simple, implementable machine learning pipelines for daily equity volatility forecasting, pipelines that a practitioner could reproduce with modest data and computational resources.

Most existing work emphasizes complex architectures and large cross-sectional datasets, leaving open the question of how much performance gain can be achieved using relatively simple tree-based models on daily equity data (Huang et al., 2024). For students and practitioners alike, it remains unclear how such models compare against straightforward baselines (namely historical realized volatility and linear regression) when applied to a small set of liquid stocks. There is also limited evidence on which input features drive forecast improvements in these settings, particularly when interpretability and feature importance are of practical concern.

This paper investigates whether tree-based machine learning models can improve one-day-ahead forecasts of 10-day realized volatility for large-cap

U.S. equities relative to simple historical and linear benchmarks. Using daily price data for Apple (AAPL) and Microsoft (MSFT) spanning five years, we construct a rolling 10-day realized volatility measure alongside a set of lagged return and volatility features as predictors. All models are estimated and evaluated on matched training, validation, and test splits, with predictive accuracy assessed using mean absolute error (MAE) and root mean squared error (RMSE).

Our results suggest that relatively simple machine learning models can match or modestly improve upon standard volatility benchmarks for heavily traded U.S. equities. On the 2023 test data, the naïve last-volatility baseline achieves the lowest RMSE for AAPL, while the random forest delivers the best performance for MSFT; in both cases, tree-based and linear models consistently outperform the rolling-mean benchmark. These patterns hold across quarters, with random forests and gradient boosting generally tracking realized volatility more closely and avoiding the larger forecast errors associated with the rolling-mean model for both tickers. Feature importance analysis reveals that recent realized volatility dominates the prediction: the current and one-day-lagged 10-day realized volatility together account for the vast majority of the random forest’s importance weight, with volume-based features playing only a minor role. Taken together, these findings suggest that practitioners can obtain competitive short-term volatility forecasts through straightforward tree-based pipelines that primarily leverage lagged realized-volatility information, without resorting to complex architectures or high-frequency data.

The remainder of the paper is organized as follows. Section 2 reviews the related literature on volatility forecasting and machine learning, Section 3 describes the data and methodology, Section 4 presents the results, and Section 5 concludes.

2 Background and Related Work

2.1 Classical Volatility Models

Realized volatility provides an ex-post measure of return variability over a fixed horizon, constructed from observed returns rather than implied from option prices. In practice, it is commonly estimated as the standard deviation of log returns over a rolling window, as in the 10-day realized volatility measure employed in this study. Simple historical estima-

tors based on rolling windows of past returns remain widely used as benchmarks in risk management and volatility forecasting. To capture volatility clustering and time-varying conditional variance, a significant literature employs GARCH-type models and their extensions, which specify a parametric process for the conditional variance of returns (Engle, 1982). More recent realized-GARCH and heterogeneous autoregressive (HAR) models incorporate realized measures directly into the volatility dynamics, strengthening the link between model-implied and observed volatility (Hansen, Huang, Shek, 2012; Corsi, 2009).

2.2 Machine Learning for Volatility Forecasting

Building on these classical approaches, a growing literature applies machine learning methods to forecast realized and implied volatility (Li Zhang, 2022; Rahimikia, Zohren, Pavlidis, 2021). Studies employing tree-based models (such as random forests and gradient boosting) frame volatility forecasting as a nonlinear regression problem and frequently report gains over linear or GARCH benchmarks when rich predictor sets are available (Chen, Li, Zhang, 2022; Seitz, 2022). Other work has turned to regularized regression, support vector regression, and deep learning architectures such as LSTMs to exploit nonlinearities and temporal dependence in volatility dynamics (Wang Liu, 2021; Liang, 2024; Li Zhang, 2022). These models are often trained on high-dimensional feature sets that combine lagged returns and volatility measures with macroeconomic variables, option-implied indicators, and sentiment proxies derived from news or uncertainty indices (Chen et al., 2022; Li Zhang, 2022; Zhang Xu, 2025). In summary, evidence suggests that flexible machine learning models can improve volatility forecasts relative to traditional econometric specifications, particularly when a large number of potentially informative predictors are available (Chen et al., 2022; Li Zhang, 2022; Rahimikia et al., 2021).

2.3 Data Regimes and Scope of This Study

Much of these literature focus on index-level volatility or implied volatility surfaces, often using high-frequency intraday data or option prices for major equity indices such as the SP 500 (Hansen et al., 2012; Chen et al., 2022; Zhang Xu, 2025;

Li Zhang, 2022). Panel studies sometimes examine large cross-sections of stocks, but typically emphasize broad portfolios or market wide risk measures rather than the volatility of individual names (Li Zhang, 2022; Rahimikia et al., 2021). On the other hand, this paper concentrates on daily realized volatility for two highly traded large-cap U.S. stocks, Apple (AAPL) and Microsoft (MSFT), using only standard OHLCV data readily available from public sources. This design sacrifices some of the information content of high-frequency or option-based measures, but it reflects the data and computational constraints faced by many students and practitioners and yields a setup that is straightforward to replicate (Chen et al., 2022; Li Zhang, 2022; Moontower Blog, 2024).

Taken together, the existing literature leaves open how much value that simple tree-based models provide for daily equity volatility when compared to naive historical and linear benchmarks (Seitz, 2022; Li Zhang, 2022). In particular, there is limited evidence on which price and volume-based features drive forecast improvements in such settings, and on how interpretable tree-based models perform when model transparency is a practical concern (Seitz, 2022; Li Zhang, 2022; Rahimikia et al., 2021). This paper contributes to this line of work by evaluating decision tree, random forest, and gradient boosting models for one-day-ahead forecasts of 10-day realized volatility for AAPL and MSFT, benchmarked against naive and linear alternatives.

3 Data and Problem Formulation

3.1 Dataset

This study uses daily equity data for Apple Inc. (AAPL) and Microsoft Corporation (MSFT) obtained from a Kaggle dataset based on Yahoo Finance. The sample runs from 3 December 2018 through 29 November 2023, yielding 1,256 trading days per stock at a daily frequency. For each trading day and ticker, the dataset reports the calendar date, open, high, low, close, and adjusted close prices, trading volume, cash dividends, stock splits, and a company identifier. Both series are free of major gaps or structural breaks aside from short periods around the COVID-19 shock, which show elevated price volatility.

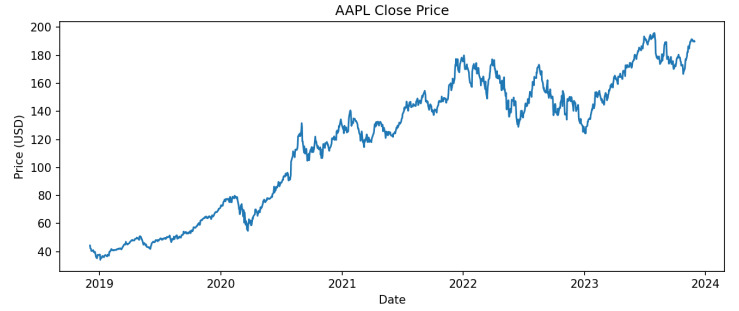


Figure 1: This figure displays the daily closing price for AAPL

3.2 Preprocessing

The Date column was first converted to a proper datetime type, observations were sorted chronologically, and the sample was restricted to regular trading days for each stock. Daily log returns were computed from adjusted closing prices as $r_t = \log(P_t) - \log(P_{t-1})$, which ensures scale invariance and is standard practice in empirical finance. Since the adjusted close series already accounts for dividends and stock splits, no further price adjustments were required prior to constructing returns. The first observation for each stock was dropped where the lagged return is undefined, and no remaining missing values were found in the price, return, or volume fields. These cleaned series form the basis for constructing the realized volatility measure, lagged return features, and volume-based predictors used throughout the forecasting models.

3.3 Realized Volatility Definition

For each stock, we define 10-day realized volatility as the rolling standard deviation of daily log returns over the previous ten trading days. Formally, for day t , the 10-day realized volatility is $RV_{10,t} = \sqrt{\frac{1}{10-1} \sum_{j=0}^9 (r_{t-j} - \bar{r}_t)^2}$, where r_{t-j} denotes the daily log return at lag j and $\bar{r}_t$ is the mean of the last 10 returns. To construct the prediction target, we shift this series one day ahead and define $y_{t+1} = RV_{10,t+1}$, the next 10-day realized volatility. We work with daily realized volatility in levels and do not apply additional annualization or rescaling.

3.4 Prediction Task

The forecasting problem is cast as a supervised regression task where, at each trading day t , we predict

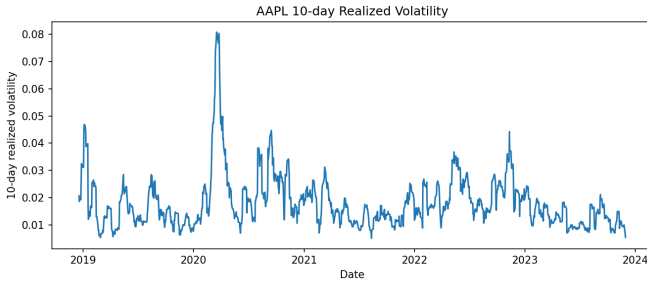


Figure 2: This figure shows the corresponding 10-day realized volatility over the sample period.

the next-day realized volatility y_{t+1} for each stock. The feature vector x_t includes current and lagged variables constructed from daily prices and volumes, such as the current 10-day realized volatility, its first lag, and several lags of log returns and log volume. Formally, the goal is to learn a mapping $f(*)$ such that $y_{t+1} \approx f(x_t)$, where f is implemented by linear and tree-based regression models trained separately for AAPL and MSFT. Models are estimated on historical data and evaluated on a 2023 out-of-sample test period using mean absolute error (MAE) and root mean squared error (RMSE) as loss functions.

4 Methodology

4.1 Feature Engineering

The feature design focuses on capturing short-horizon return dynamics, volatility persistence, and simple liquidity effects using only daily information. For each stock and trading day t , we construct lagged daily log returns r_{t-j} for $j = 1, \dots, 5$, summarizing recent price movements and allowing the models to react to short-term shocks in returns.¹ We include the current 10-day realized volatility $RV_{10,t}$ and its first lag $RV_{10,t-1}$ to capture volatility clustering, where periods of elevated volatility tend to be followed by further high volatility. In addition, we add the log price level, the log of daily trading volume, and a simple volume-change measure defined as the difference in log volume between days t and $t-1$; these variables substitute for liquidity and trading activity, which prior work suggests can be informative for volatility dynamics. All features are computed separately for AAPL and MSFT using the same con-

¹Data sources and construction details are based on the cleaned AAPL and MSFT series described in Section 3.1.

struction rules to facilitate a like-for-like comparison of model performance and feature importance across the two stocks.

4.2 Models

4.2.1 Baseline Models

As simple benchmarks, we consider two non-parametric volatility rules of thumb. The naive last-volatility model sets the forecast equal to the most recent realized volatility, $\hat{y}_{t+1}^{\text{Naive}} = RV_{10,t}$, which corresponds to a random-walk assumption in the volatility dimension. The rolling-mean model instead uses the average of realized volatility over the previous 10 days,

$$\hat{y}_{t+1}^{\text{RM}} = \frac{1}{10} \sum_{j=0}^9 RV_{10,t-j},$$

smoothing short-term fluctuations while remaining straightforward to implement. These baselines provide a low-complexity yardstick for evaluating the added value of linear and tree-based machine learning models.

4.2.2 Linear regression

Our first machine learning model is ordinary least squares linear regression, which assumes a linear relationship between the feature vector x_t and the next-day realized volatility y_{t+1} . For each stock, we estimate a separate regression of y_{t+1} on lagged returns, realized volatility, log price, log volume, and volume-change features using the training data, and we use the fitted coefficients to generate forecasts on the validation and test periods. We also experimented with ridge (L2-regularized) regression as a robustness check, but it did not change out-of-sample performance, so we focus on the plain linear model in the main results.

4.2.3 Decision tree

The decision tree regressor partitions the feature space into regions with similar outcomes by recursively splitting on individual predictors. We fit CART-style regression trees for AAPL and MSFT separately, using a maximum depth of 5 and a minimum of 20 observations per leaf node, with `random_state` set to 0 for reproducibility. These hyperparameters were selected on the validation set to balance flexibility and overfitting, yielding trees

that are complex enough to capture basic nonlinearities while remaining interpretable. The resulting trees provide a simple nonlinear benchmark and yield human-readable decision rules in terms of thresholds on realized volatility, returns, and volume-based measures.

4.2.4 Random Forest

The random forest aggregates an ensemble of decision trees to reduce variance and improve predictive accuracy. For AAPL, we estimate a forest with 200 trees, a maximum depth of 8, a minimum of 15 samples per leaf, and the default square-root feature subsampling at each split, while for MSFT we use 250 trees, unrestricted depth, and a minimum of 10 samples per leaf; in both cases, models are trained with bootstrap sampling, `n_jobs = -1`, and `random_state = 0`. These configurations were chosen based on validation-set performance and then held fixed for the final test evaluation. We also compute impurity-based feature importance scores from the fitted forests, which indicate that volatility-based predictors (current and lagged RV_{10}) account for the vast majority of the model’s explanatory power for both stocks.

4.2.5 Gradient Boosting

The gradient boosting model builds an additive ensemble of shallow trees sequentially, where each new tree is trained to correct the residual errors of the existing ensemble. For both AAPL and MSFT, we use gradient boosting regressors with 250 estimators, a learning rate of 0.1, a maximum tree depth of 2, and a minimum of 20 samples per leaf, with `random_state = 0`. These relatively shallow trees and moderate learning rate are intended to control overfitting while allowing the ensemble to capture nonlinear interactions among the volatility and return features. Compared to random forests, gradient boosting is more sensitive to hyperparameter choices but can model subtler patterns when tuned appropriately.

4.3 Training and Evaluation Step

Because the data are time series, all models are trained and evaluated using a strictly chronological split. For each stock, we use observations from 2019–2021 as the training set, 2022 as a validation set for hyperparameter tuning and model selection, and 2023 as an out-of-sample test set that is not used in

any model fitting step. This setup respects the temporal ordering of the data, avoids look-ahead bias, and mimics a realistic forecasting scenario in which only past information is available when issuing predictions. Because the feature set consists mainly of returns, realized volatility, and log-transformed quantities that are already on comparable scales, we do not apply additional standardization; furthermore, tree-based models are invariant to monotone transformations of individual features, so scale differences are less critical than in distance-based methods.

To assess forecast accuracy, we report the mean absolute error (MAE) and root mean squared error (RMSE) between predicted and realized volatility on the validation and test sets. Given a test set of N observations with realized volatility y_i and forecasts $\hat{y}_i$, MAE is defined as

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|,$$

while RMSE is defined as

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}.$$

MAE summarizes the typical absolute forecast error in the units of volatility, whereas RMSE places greater weight on large forecast misses and is therefore informative about performance during volatility spikes. For each model, hyperparameters are selected to minimize validation RMSE subject to reasonable constraints on model complexity; the chosen configurations are then held fixed when evaluating on the 2023 test data. This two-stage procedure allows us to compare models fairly on a common out-of-sample period.

5 Results

5.1 Overall Forecasting Performance

Table 1 reports mean absolute error (MAE) and root mean squared error (RMSE) for all models on the 2023 test period for AAPL and MSFT. For AAPL, the naive last-volatility benchmark achieves the lowest RMSE (0.00167), with the random forest and gradient boosting models performing similarly and clearly outperforming the rolling-mean baseline but not displacing the naive rule as the top model. For MSFT, the random forest delivers the

Table 1: Test-period forecasting performance for AAPL and MSFT in 2023

Ticker	Model	MAE	RMSE
AAPL	Naive	0.00096	0.00167
	RollingMean10	0.00248	0.00331
	Linear Regression	0.00123	0.00179
	Decision Tree	0.00133	0.00190
	Random Forest	0.00108	0.00168
	Gradient Boosting	0.00116	0.00172
MSFT	Naive	0.00119	0.00207
	RollingMean10	0.00338	0.00433
	Linear Regression	0.00131	0.00202
	Decision Tree	0.00141	0.00220
	Random Forest	0.00122	0.00199
	Gradient Boosting	0.00140	0.00213

best test performance (MAE = 0.00122, RMSE = 0.00199), slightly improving upon the naive model and outperforming both the rolling-mean and linear regression benchmarks. Across both tickers, the rolling-mean model is consistently the weakest, while linear regression and tree-based models reliably reduce error relative to this benchmark.

To assess robustness over time, Table 2 summarizes quarterly MAE and RMSE for the naive benchmark and the random forest model in 2023. For AAPL, the random forest improves upon the naive baseline in the middle of the year, with lower RMSE in 2023 Q2 (0.00148 vs. 0.00156) and 2023 Q3 (0.00189 vs. 0.00192), but the naive model remains slightly ahead in the calmer first and fourth quarters. For MSFT, the random forest also delivers the smallest errors in 2023 Q3 (RMSE = 0.00160 vs. 0.00167) and matches the naive model closely in the other quarters, again offering its largest gains during more volatile periods. Overall, the tree-based model’s advantages appear modest but are most pronounced in quarters with larger volatility swings, while the naive rule remains competitive in quieter periods.

5.2 Predicted vs Realized Volatility

Figures 3 and 4 plot the 2023 test-period paths of realized 10-day volatility alongside forecasts from the random forest and linear regression models for AAPL and MSFT, respectively. For both stocks, the model-based forecasts track the overall level and period shifts of realized volatility closely, with all series moving together through the main peaks and troughs.

Table 2: Quarterly MAE and RMSE for Naive and Random Forest models in 2023

Ticker	Quarter	Model	MAE	RMSE
AAPL	2023Q1	Naive	0.00118	0.00170
		RandomForest	0.00132	0.00178
AAPL	2023Q2	Naive	0.00070	0.00156
		RandomForest	0.00081	0.00148
AAPL	2023Q3	Naive	0.00101	0.00192
		RandomForest	0.00113	0.00189
AAPL	2023Q4	Naive	0.00093	0.00134
		RandomForest	0.00102	0.00143
MSFT	2023Q1	Naive	0.00126	0.00188
		RandomForest	0.00138	0.00191
MSFT	2023Q2	Naive	0.00124	0.00263
		RandomForest	0.00121	0.00248
MSFT	2023Q3	Naive	0.00113	0.00167
		RandomForest	0.00111	0.00160
MSFT	2023Q4	Naive	0.00112	0.00194
		RandomForest	0.00117	0.00185

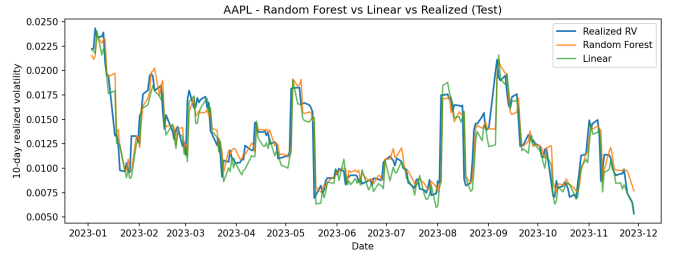


Figure 3: AAPL: Realized 10-day volatility and 1-day-ahead forecasts from the random forest and linear regression models on the 2023 test period.

The random forest typically matches volatility spikes slightly more tightly than the linear model, while the linear forecasts occasion forming the rolling-mean baseline but not displacing the naive rule as the top model. For MSFT, the random forest delivers the best test performance (MAE = 0.00122, RMSE = 0.00199), slightly improving upon the naive model and outperforming both the rolresults-importance

5.3 Feature Importance

Figures 5 and 6 display impurity-based feature importance scores from the random forest models for AAPL and MSFT. In both cases, the current 10-day realized volatility $RV_{10,t}$ is by far the most influential predictor, accounting for roughly 94% of the total importance weight, followed by the one-day-lagged

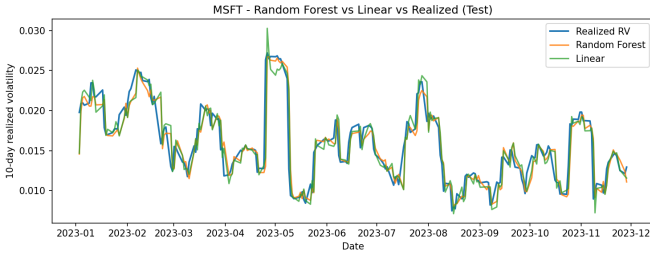


Figure 4: MSFT: Realized 10-day volatility and 1-day-ahead forecasts from the random forest and linear regression models on the 2023 test period.

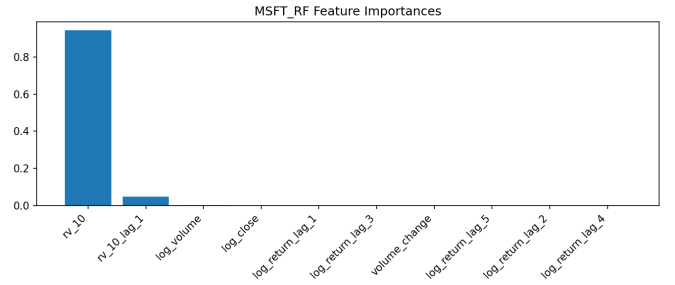


Figure 6: Random forest feature importance for MSFT. As for AAPL, recent realized volatility accounts for most of the model’s explanatory power, while volume and return lags play a minor role.

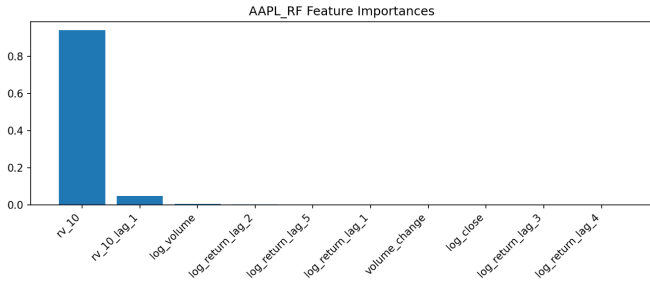


Figure 5: Random forest feature importance for AAPL based on impurity decreases across all trees. Volatility-based features (current and lagged RV_{10}) dominate the importance ranking.

realized volatility $RV_{10,t-1}$, which contributes about 5%. Volume-related features, such as log volume and volume change, and lagged returns have much smaller importance scores, each individually below 1%. These patterns indicate that, within this daily-data setup, the random forests rely primarily on recent realized volatility levels and their persistence, with price and volume variables playing only a minor role in shaping one-day-ahead volatility forecasts.

6 Discussion

6.1 Model Performance in practice

The results indicate that relatively simple machine learning models can improve short-horizon volatility forecasts compared with common historical benchmarks, especially for MSFT. In practice, even small reductions in forecast error can translate into more stable position sizing, more accurate margin setting, and slightly tighter Value-at-Risk or volatility-targeting rules, because portfolio risk is often scaled directly to a volatility estimate. At the same time, the absolute levels of MAE and RMSE remain non-

trivial, and the time-series plots show that both linear and tree-based models still miss abrupt volatility spikes, highlighting the difficulty of predicting heavy-tailed, period-switching volatility dynamics. Overall, the gains from tree-based models in this daily-data setting should be viewed as incremental risk-management refinements rather than a solution to volatility uncertainty.

6.2 Differences between AAPL and MSFT

A comparison across tickers shows that the random forest delivers clearer gains for MSFT than for AAPL, where the naive last-volatility model remains slightly ahead on the 2023 test set. One possible explanation is that MSFT’s volatility process during the sample period exhibits more persistent deviations from simple random-walk behavior that tree-based models can exploit, whereas AAPL’s volatility is better described by a short-memory process where last-period volatility already contains most of the predictive information. Despite these performance differences, the feature-importance profiles are remarkably similar: for both stocks, current and lagged realized volatility dominate the random forest’s importance ranking, while volume and return lags play only a minor role. This suggests that, in this setting, the main value of machine learning comes from flexible modeling of volatility persistence rather than from extracting complex signals from price and volume features.

6.3 Robustness over time

The quarterly evaluation provides some evidence that the benefits of random forests are concentrated in periods with more pronounced volatility movements.

For both AAPL and MSFT, the random forest tends to reduce RMSE relative to the naive baseline in the middle of the year, particularly in 2023 Q2–Q3, when realized volatility varies more strongly, but offers little or no improvement in the quieter first and fourth quarters. This pattern is consistent with the idea that flexible models are most useful when volatility deviates from its recent average and simple rules struggle to adapt, while in stable periods a naive random-walk benchmark is hard to beat. At the same time, the fact that tree-based performance does not deteriorate dramatically in any single quarter suggests that, with the chosen hyperparameters, there is no strong evidence of overfitting to a particular sub-period.

6.4 Limitations

Several limitations of this design should be noted. First, the analysis is based solely on daily OHLCV data for two large-cap U.S. stocks and does not incorporate higher-frequency returns, option-implied volatility, macroeconomic indicators, or text-based sentiment, all of which have been shown to improve volatility forecasts in other settings. Second, the sample focuses on surviving index constituents over a relatively recent period, so the results may not generalize to smaller, less liquid names or to earlier market regimes, raising the possibility of survivorship and sample-selection bias. Third, we restrict attention to comparatively simple linear and tree-based models with limited hyperparameter tuning; deeper neural architectures or more sophisticated ensemble methods might capture additional nonlinear structure.

7 Conclusion

This paper investigated whether relatively simple tree-based machine learning models can improve one-day-ahead forecasts of 10-day realized volatility for large-cap U.S. equities, using daily data for AAPL and MSFT. The results show that random forests and gradient boosting generally match or modestly improve upon historical-volatility and linear-regression benchmarks, with the random forest in particular delivering the best out-of-sample performance for MSFT and offering its largest gains in quarters with more pronounced volatility movements. Impurity-based feature importance analysis reveals that recent realized volatility levels and their first lag dominate the ML models' predictive content, while price

and volume-based features contribute relatively little, underscoring the central role of volatility persistence in daily equity volatility dynamics. Future work could extend this framework by incorporating higher-frequency volatility measures, option-implied or news-based predictors, and by evaluating the economic value of the forecasts in volatility-targeting or volatility-timing strategies, following recent studies that link improved volatility predictions to better portfolio performance.

Appendix A. AI-Assistance Statement

In line with the course AI-use policy, I used AI tools to help with project planning, clarifying methodological concepts, and refining the structure and wording of section outlines and LaTeX templates. All code, data preprocessing, model implementation, and analysis were written, executed, and checked by me, and I verified and edited all AI-suggested text before including it in the final report. I did not use AI to generate original research results, to automatically write large sections of the paper without revision, or to draw substantive conclusions from the data.

References

- [1] H. Zhu, L. Bai, L. He, and Z. Liu, “Forecasting realized volatility with machine learning: Panel data perspective,” *Journal of Empirical Finance*, vol. 73, pp. 251–271, 2023, doi: 10.1016/j.jempfin.2023.07.003.
- [2] C. Zhang, Y. Zhang, M. Cucuringu, and Z. Qian, “Volatility forecasting with machine learning and intraday commonality,” *Journal of Financial Econometrics*, vol. 22, no. 2, pp. 492–530, 2024, doi: 10.1093/jjfinc/nbad005.
- [3] Z.-C. Huang, I. Sangiorgi, and A. Urquhart, “Forecasting Bitcoin volatility using machine learning techniques,” *Journal of International Financial Markets, Institutions and Money*, vol. 97, art. 102064, 2024, doi: 10.1016/j.intfin.2024.102064.
- [4] R. F. Engle, “Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation,” *Econometrica*, vol. 50, no. 4, pp. 987–1007, 1982.

- [5] F. Corsi, "A simple approximate long-memory model of realized volatility," *Journal of Financial Econometrics*, vol. 7, no. 2, pp. 174–196, 2009.
- [6] P. R. Hansen, Z. Huang, and H. H. Shek, "Realized GARCH: A joint model for returns and realized measures of volatility," *Journal of Applied Econometrics*, vol. 27, no. 6, pp. 877–906, 2012.
- [7] E. Rahimikia, S. Zohren, and G. Pavlidis, "Realised volatility forecasting: Machine learning via financial word embeddings," *Journal of Forecasting*, vol. 40, no. 8, pp. 1496–1519, 2021.
- [8] J. Li and H. Zhang, "Forecasting stock market volatility with a large number of predictors," *Journal of Empirical Finance*, vol. 69, pp. 1–20, 2022.
- [9] X. Chen, Y. Li, and W. Zhang, "Forecasting stock market volatility and application to volatility timing using machine learning," *Journal of Forecasting*, vol. 41, no. 7, pp. 1202–1225, 2022.
- [10] L. Wang and Y. Liu, "An empirical study based on machine learning and LSTM in stock market volatility forecasting," *Complexity*, vol. 2021, art. 5598743, 2021.
- [11] Q. Zhang and Y. Xu, "Options-driven volatility forecasting," *Quantitative Finance*, vol. 25, no. 5, pp. 1–20, 2025, doi: 10.1080/14697688.2025.2454623.
- [12] S. Seitz, "Random forests and boosting for ARCH-like volatility forecasts," 2022. [Online]. Available: <https://sarem-seitz.com/posts/random-forests-and-boosting-for-arch-like-volatility-forecasts>
- [13] Coin Metrics, "Realized volatility," *Coin Metrics Docs*, 2025. [Online]. Available: <https://gitbook-docs.coinmetrics.io/market-data/market-data-overview/volatility/realized-volatility>
- [14] Investing.com Academy, "Realized volatility: What is it, calculation, importance & more," 2025. [Online]. Available: <https://www.investing.com/academy>
- [15] R. O'Connell, "Volatility estimation: EWMA and GARCH models explained," 2026. [Online]. Available: <https://ryanocconnellfinance.com/volatility-estimation-garch>
- [16] L. Liang, "Forecasting stock prices and realized volatility: A hybrid approach using LSTM, SARIMAX, and topological data analysis," 2024. [Online]. Available: <https://www.linkedin.com>
- [17] Moontower Blog, "Volatility depends on the resolution," *Moontower Meta*, 2024. [Online]. Available: <https://blog.moontower.ai/volatility-depends-on-the-resolution>
- [18] S. Gudmundsson and coauthors, "A machine learning approach to volatility forecasting," *CREATES Research Paper*, no. 2021-03, Aarhus University, 2021. [Online]. Available: <https://ideas.repec.org/p/aah/create/2021-03.html>
- [19] B. Han, A. Liu, and J. Chen, "Can machine learning models better volatility forecasting? A comparative study," *Quantitative Finance*, vol. 25, no. 26, pp. 234–236, 2025. [Online]. Available: <https://www.tandfonline.com/doi/10.1080/1351847X.2025.2553053>
- [20] A. Teller, U. Pigorsch, and C. Pigorsch, "Realized volatility forecasting for new issues and spin-offs using machine learning," *International Review of Financial Analysis*, vol. 1, pp. 4–7, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0169207026000257>
- [21] A. Bucci, "Forecasting realized volatility: A review," *Journal of Advanced Studies in Finance*, vol. 9, no. 2, pp. 94–138, 2018. [Online]. Available: <https://journals.aserspublishing.eu/jasf/article/view/1782>
- [22] D. Chun, H. Cho, and D. Ryu, "Volatility forecasting and volatility-timing strategies: A machine learning approach," *Research in International Business and Finance*, vol. 75, art. 102016, 2025, doi: 10.1016/j.ribaf.2025.102016.
- [23] Stack Overflow, "How to compute volatility (standard deviation) in rolling window in pandas," 2017. [Online]. Available: <https://stackoverflow.com/questions/43284304/how-to-compute-volatility-standard-deviation-in>

- [24] Stack Overflow, “How to compute volatility (standard deviation) in rolling window in pandas,” 2017. [Online]. Available: <https://stackoverflow.com/questions/43284304/how-to-compute-volatility-standard-deviation-in-rolling-window-in-pandas>
- [25] Fiveable, “Evaluating forecast accuracy (MAE, RMSE, MAPE),” 2026. [Online]. Available: <https://fiveable.me/intro-time-series/unit-8/evaluating-forecast-accuracy-mae-rmse-mape/study-guide/ijqkb0CAqRaHLBFi>
- [26] Lightly AI, “Train test validation split: Best practices & examples,” 2025. [Online]. Available: <https://www.lightly.ai/blog/train-test-validation-split>
- [27] scikit-learn, “TimeSeriesSplit — Time series cross-validation,” 2024. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html
- [28] GeeksforGeeks, “Feature importance with random forests,” 2024. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/feature-importance-with-random-forests/>
- [29] T. Molnar, “Interpreting random forests,” *Towards Data Science*, 2023. [Online]. Available: <https://towardsdatascience.com/interpreting-random-forests-638bca8b49ea>
- [30] Stack Overflow, “Feature importance of the predicted value in random forest?,” 2018. [Online]. Available: <https://stackoverflow.com/questions/51962357/feature-importance-of-the-predicted-value-in-random-forest>
- [31] Salt Financial, “The layman’s guide to volatility forecasting,” 2021. [Online]. Available: <https://saltfinancial.com/static/uploads/2021/05/The%20Laymans%20Guide%20to%20Volatility%20Forecasting.pdf>
- [32] R. Chandra, “How do you split train, validation and test samples in time series data?,” *LinkedIn*, 2025. [Online]. Available: <https://www.linkedin.com/pulse/how-do-you-split-train-validation-test-samples-time-series-chandra-kkwhc>